

Name: Mimanshu Gahlaut

UID: 24BAI70038

Course: BE-CSE (AI&ML)

Subject: Database Management System

Experiment 8: Stored Procedures

1. Aim of the Session

To understand the design and implementation of stored procedures in PL/SQL, focusing on the use of different parameter modes (IN, OUT, and INOUT) to encapsulate business logic and facilitate modular programming within a database.

2. Software Requirements:

- Database Management System:
 - Oracle Database Express Edition (Oracle XE)
 - PostgreSQL Database
- Database Administration Tool / Client Tool:
 - Oracle SQL Developer (for Oracle XE)
 - pgAdmin (for PostgreSQL)

3. Objective of the Session

To create and execute a parameterized stored procedure that processes data from an employee table, utilizes multiple parameter types to return calculated results and execution status, and demonstrates the invocation of procedures using anonymous blocks.

4. Practical / Experiment Steps

The work was carried out through the following activities:

1. **Table Creation:** Created an `employees` table with fields such as employee ID, name, gender, and salary.

2. **Procedure Development:** Implemented a stored procedure named `count_employees_by_gender`.
3. **Parameter Handling:** Used IN parameter for input (gender), OUT parameter for returning count, and INOUT parameter for status tracking.
4. **Query Integration:** Applied `SELECT COUNT(*) INTO` to fetch filtered results.
5. **Execution Block:** Used an anonymous block to call the procedure and display results.

5. Procedure of the Practical

Execution was performed in the following order:

- Connected to the PostgreSQL database environment.
- Created and populated the employees table with sample data.
- Defined a stored procedure with appropriate parameter modes.
- Wrote logic to count employees based on gender.
- Updated the status parameter to indicate successful execution.
- Declared variables inside an anonymous block.
- Called the procedure using the CALL statement.
- Displayed output using RAISE NOTICE.
- Verified results with actual table data.

6. I/O Analysis (Input / Output Analysis)

Input Queries

SQL

```
CREATE TABLE employe(  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(20),  
    gender VARCHAR(20),  
    salary NUMERIC(10, 2)  
);
```

```

1 CREATE TABLE employe(
2     emp_id INT PRIMARY KEY,
3     emp_name VARCHAR(20),
4     gender VARCHAR(20),
5     salary NUMERIC(10, 2)
6 );
7

```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 105 msec.

```

INSERT INTO employe (emp_id, emp_name, gender, salary) VALUES
(101, 'Amitabh', 'Male', 30000),
(102, 'Annu', 'Female', 55000),
(103, 'Riya', 'Female', 45000),
(104, 'Amrit', 'Male', 59000),
(105, 'Anjali', 'Female', 66000);

```

```

8 INSERT INTO employe (emp_id, emp_name, gender, salary) VALUES
9 (101, 'Amitabh', 'Male', 30000),
10 (102, 'Annu', 'Female', 55000),
11 (103, 'Riya', 'Female', 45000),
12 (104, 'Amrit', 'Male', 59000),
13 (105, 'Anjali', 'Female', 66000);
14

```

Data Output Messages Notifications

INSERT 0 5

Query returned successfully in 124 msec.

```

CREATE OR REPLACE PROCEDURE get_employe_count_by_gender (
    IN IN_GENDER VARCHAR(20),
    OUT OUT_COUNT INT,
    INOUT STATUS VARCHAR(20)
)
AS
$$
BEGIN
    SELECT COUNT(*) INTO OUT_COUNT
    FROM employe
    WHERE gender = IN_GENDER;

    STATUS := 'SUCCESS';
END;
$$ LANGUAGE PLPGSQL;

```

```

15 CREATE OR REPLACE PROCEDURE get_employe_count_by_gender (
16     IN IN_GENDER VARCHAR(20),
17     OUT OUT_COUNT INT,
18     INOUT STATUS VARCHAR(20)
19 )
20 AS
21 $$
22 BEGIN
23     SELECT COUNT(*) INTO OUT_COUNT
24     FROM employe
25     WHERE gender = IN_GENDER;
26
27     STATUS := 'SUCCESS';
28 END;
29 $$ LANGUAGE PLPGSQL;
30

```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 93 msec.

```

DO
$$
DECLARE
    GEN VARCHAR(20) := 'Female';
    COUNT_OF_EMPLOYE INT;
    STATUS VARCHAR := 'PENDING';

BEGIN
    CALL get_employe_count_by_gender(GEN, COUNT_OF_EMPLOYE, STATUS);

    RAISE NOTICE 'COUNT OF % EMPLOYEES IS % AND STATUS IS %',
    GEN, COUNT_OF_EMPLOYE, STATUS;
END;
$$

```

```

31 DO
32 $$
33 DECLARE
34     GEN VARCHAR(20) := 'Female';
35     COUNT_OF_EMPLOYE INT;
36     STATUS VARCHAR := 'PENDING';
37
38 BEGIN
39     CALL get_employe_count_by_gender(GEN, COUNT_OF_EMPLOYE, STATUS);
40
41     RAISE NOTICE 'COUNT OF % EMPLOYEES IS % AND STATUS IS %',
42     GEN, COUNT_OF_EMPLOYE, STATUS;
43 END;
44 $$

```

Data Output Messages Notifications

NOTICE: COUNT OF Female EMPLOYEES IS 3 AND STATUS IS SUCCESS
DO

Query returned successfully in 100 msec.

7. Learning Outcome

- Understood how to encapsulate SQL logic into reusable procedures.
- Learned the use of **IN, OUT, and INOUT parameters** in stored procedures.
- Gained knowledge of using SELECT INTO for assigning query results.
- Learned how to display output using RAISE NOTICE.